Reinforcement Learning: Домашнее задание

1 Цель домашнего задания

В этом задании мы попрактикуемся с применением Reinforcement Learning (RL) к улучшению агентских способностей модели – вызову функций. На вход модель будет получать запрос пользователя и описания доступных инструментов. Здесь она должна выбрать нужные функции и передать правильные аргументы, чтобы удовлетворить запрос пользователя. Также, есть случаи, когда модель не должна делать вызов функций. Например, когда подходящего инструмента нету, или пользователь не дал достаточно данных для аргументов нужной функции (в таком случае, например, модель должна переспросить)

Ваша цель – максимально улучшить качество исходной модели на фиксированном бенчмарке. Для этого нужно самостоятельно подготовить данные, спроектировать награду, провести несколько экспериментов и объяснить полученные результаты.

2 Сетап

1. **Модель:** Qwen/Qwen3.5-2B. Работаем только с этой моделькой – менять ее нельзя (также нельзя брать другие чекпоинты этой модельки, например).
2. **Обучение:** Разрешается использовать только алгоритмы RL. То есть SFT делать нельзя. При этом можно использовать всякие параметро-эффективные методы по типу LoRa (но тут не забудьте объединить веса модельки перед замером качества)
3. **Данные:** Team-ACE/ToolACE. Загрузку, разбор и преобразование данных вы реализуете самостоятельно. Для обучения используйте только ToolACE. Другие источники данных использовать нельзя.
4. **Бенчмарк:** Замеряемся на семпле примеров из Berkeley Function Calling Leaderboard (BFCL). Подробнее про него распишу ниже, но у вас будет дано только 350 примеров от исходного бенчмарка. Итоговая метрика – `overall` из `evaluate.py`.
5. **Фреймворк:** TRL. В Docker-образе уже установлены TRL 1.16.0, Transformers, PEFT, vLLM и `bfcl-eval`. При этом можно ставить свою версию TRL и других библиотек, но скорее всего придется долго разбираться с зависимостями – поэтому рекомендую использовать готовый образ.

2.1 Сборка и запуск окружения

На ваших машинках работайте в какой-нибудь своей папке, положите туда `Dockerfile`, `evaluate.py` и `bfcl.jsonl`. Потом можно вот так собрать образ и запустить контейнер. `device=0` не забудьте поменять на тот номер GPU, с которым вы работаете.

```
docker build -t rlvr-homework .
docker run --rm -it --gpus device=0 --ipc=host \
  -v "$PWD":/workspace -v rlvr-hf-cache:/cache rlvr-homework
```

В контейнере текущая папка - `/workspace`, удобнее работать в ней.

Сначала оцените исходную модель:

```
python3 evaluate.py Qwen/Qwen3.5-2B --output results/base
```

После обучения оцените полный Hugging Face чекпоинт:

```
python3 evaluate.py ./checkpoints/final --output results/final
```

3 Варианты GRPO: что именно меняется

Вспомним из лекции GRPO, а также рассмотрим его некоторые модификации.

3.1 Обозначения и напоминание GRPO

Используем следующие обозначения

- $j = 1, \ldots, B$ — индекс запроса в батче из B запросов;
- $i = 1, \ldots, G$ — индекс ответа на запрос j; для каждого запроса генерируем G ответов;
- $t = 1, \ldots, T_{j,i}$ — индекс токена в ответе i на запрос j; $T_{j,i}$ — длина этого ответа;
- $s_{j,i,t}$ — состояние: запрос и уже сгенерированный префикс ответа;
- $a_{j,i,t}$ — действие: следующий сгенерированный токен;
- $R_{j,i}$ — награда за весь ответ i на запрос j.

Таким образом, в батче $B \cdot G$ ответов.

$J(\theta)$ — целевая функция, которую максимизируем. При реализации через градиентный спуск минимизируем $\mathcal{L}(\theta) = -J(\theta)$.

Advantage. Среднее и стандартное отклонение наград считаются отдельно для каждого запроса:

$$\bar{R}_j = \frac{1}{G} \sum_{k=1}^{G} R_{j,k}, \qquad \sigma_j = \mathrm{std}(R_{j,1}, \ldots, R_{j,G}).$$

В GRPO один Advantage распространяется для всех токенов ответа:

$$\Psi_{j,i,t} = \hat{A}_{j,i}, \qquad \hat{A}^{\mathrm{GRPO}}_{j,i} = \frac{R_{j,i} - \bar{R}_j}{\sigma_j + \delta}, \qquad \delta > 0.$$

Здесь δ — небольшая константа для численной устойчивости.

Probability ratio и clipping. Ответы генерирует старая политика π_{old}. При обновлении текущей политики π_θ для каждого токена считаем:

$$\rho_{j,i,t}(\theta) = \frac{\pi_\theta(a_{j,i,t} \mid s_{j,i,t})}{\pi_{\mathrm{old}}(a_{j,i,t} \mid s_{j,i,t})}.$$

Для краткости обозначим вклад одного токена через $u_{j,i,t}(\theta)$:

$$u_{j,i,t}(\theta) = \min\Big(\rho_{j,i,t}(\theta)\hat{A}_{j,i},\ \mathrm{clip}(\rho_{j,i,t}(\theta), 1-\epsilon, 1+\epsilon)\,\hat{A}_{j,i}\Big).$$

Тогда целевая функция GRPO записывается так:

$$J_{\text{GRPO}}(\theta) = \frac{1}{B}\sum_{j=1}^{B}\frac{1}{G}\sum_{i=1}^{G}\frac{1}{T_{j,i}}\sum_{t=1}^{T_{j,i}} u_{j,i,t}(\theta).$$

Сначала усредняем токены внутри ответа, затем ответы на один запрос, затем запросы батча. Каждый ответ имеет одинаковый вес, но отдельный токен длинного ответа получает меньший коэффициент, чем токен короткого.

Для простоты здесь и далее опущен KL-штраф. В TRL его коэффициент задаётся параметром `beta`.

Если далее не указано иное, Advantage вычисляется так же, как в GRPO.

3.2 Dr. GRPO: две нормировки

Dr. GRPO меняет две вещи.

Во-первых, advantage больше не делится на стандартное отклонение наград внутри группы:

$$\hat{A}_{j,i}^{\text{Dr. GRPO}} = R_{j,i} - \bar{R}_j.$$

Во-вторых, вместо длины конкретного ответа $T_{j,i}$ используется одна фиксированная константа $T_{\max}$:

$$J_{\text{Dr. GRPO}}(\theta) = \frac{1}{BGT_{\max}}\sum_{j=1}^{B}\sum_{i=1}^{G}\sum_{t=1}^{T_{j,i}} u_{j,i,t}(\theta).$$

Здесь в $u_{j,i,t}$ подставляется $\hat{A}_{j,i}^{\text{Dr. GRPO}}$. Коэффициент при каждом токене одинаков и не зависит от фактической длины ответа.

3.3 DAPO: усреднение по токенам и раздельный clipping

В DAPO сумма вкладов токенов делится на фактическое число учитываемых токенов во всём батче:

$$J_{\text{DAPO}}(\theta) = \frac{\displaystyle\sum_{j=1}^{B}\sum_{i=1}^{G}\sum_{t=1}^{T_{j,i}} u_{j,i,t}(\theta)}{\displaystyle\sum_{j=1}^{B}\sum_{i=1}^{G} T_{j,i}}.$$

В GRPO одинаковый вес имеют ответы, а здесь — токены. Поэтому длинный ответ содержит больше слагаемых с одинаковым коэффициентом. В отличие от Dr. GRPO знаменатель зависит от фактических длин ответов.

Также DAPO использует раздельные границы клиппинга (то есть не просто эпсилон). В определении $u_{j,i,t}$ интервал заменяется на

$$[1 - \epsilon_{\text{low}},\ 1 + \epsilon_{\text{high}}].$$

В TRL нижняя граница задаётся через `epsilon`, верхняя — через `epsilon_high`.

В статье также предложена динамическая фильтрация промптов с одинаковой наградой (dynamic sampling) и специальный штраф для слишком длинных ответов (overlong reward shaping). Dynamic Sampling выкидывает "на лету"из обучения те примеры, где все награды равны либо нулю, либо

единичке (в таком случае Advantage = 0, и пример не несет обучающего сигнала). Но в TRL эта штука не реализована

3.4 GSPO: probability ratio на уровне ответа

В GRPO отношение вероятностей $\rho_{j,i,t}$ считается отдельно для каждого токена. GSPO использует одно отношение на весь ответ, нормированное по его длине:

$$\rho_{j,i}^{\text{seq}}(\theta) = \exp\left(\frac{1}{T_{j,i}}\sum_{t=1}^{T_{j,i}}\log\rho_{j,i,t}(\theta)\right).$$

Это геометрическое среднее токенных отношений вероятностей. Групповая оценка Advantage сохраняется, а clipping применяется к $\rho_{j,i}^{\text{seq}}$:

$$u_{j,i}^{\text{seq}}(\theta) = \min\Big(\rho_{j,i}^{\text{seq}}(\theta)\hat{A}_{j,i},\ \text{clip}\Big(\rho_{j,i}^{\text{seq}}(\theta), 1-\epsilon, 1+\epsilon\Big)\hat{A}_{j,i}\Big).$$

Целевая функция усредняется по ответам и запросам:

$$J_{\text{GSPO}}(\theta) = \frac{1}{B}\sum_{j=1}^{B}\frac{1}{G}\sum_{i=1}^{G}u_{j,i}^{\text{seq}}(\theta).$$

3.5 CISPO: clipping веса перед градиентом log-probability

В GRPO, если мы клиппнули токен, то его градиент равен нулю. Ребята из MiniMax решили, что так делать плохо

В CISPO ограничивается importance weight, после чего этот вес отсоединяется от графа градиентов. То есть мы не пробрасываем градиент через него, а просто используем как скаляр, который масштабирует вклад этого токена. В реализации TRL 1.16.0 с верхней границей c:

$$w_{j,i,t} = \text{stopgrad}(\min(\rho_{j,i,t}(\theta), c)).$$

Вместо clipped-вклада $u_{j,i,t}$ используется взвешенная log-probability:

$$J_{\text{CISPO}}(\theta) = \frac{\sum_{j=1}^{B}\sum_{i=1}^{G}\sum_{t=1}^{T_{j,i}} w_{j,i,t}\hat{A}_{j,i}\log\pi_\theta(a_{j,i,t}\mid s_{j,i,t})}{\sum_{j=1}^{B}\sum_{i=1}^{G}T_{j,i}}.$$

При дифференцировании $w_{j,i,t}$ считается константой. Поэтому ограничение веса само по себе не обнуляет градиент $\log\pi_\theta$. Усреднение по токенам такое же, как в DAPO.

4 Процесс решения

4.1 Подготовка данных

Здесь нужно реализовать скрипт подготовки данных из ToolACE в нужный вам формат для обучения. Например, дефолтным способом обучения вызову функций является раскручивание примера, когда мы берем каждый диалог (пример), нарезаем его на префиксы + ответ ассистента. Например был диалог (сообщение пользователя 1 -> ответ модели 1 -> сообщение пользователя 2

-> ответ модели 2), тогда мы получим два примера для RL (1) промпт: (сообщение пользователя 1), правильное решение: (ответ модели 1) (2) промпт: (сообщение пользователя 1 -> ответ модели 1 -> сообщение пользователя 2), правильное решение: (ответ модели 2). В таком случае моделька в RL получает на вход промпт, а должна вызвать те же функции, что и в исходном диалоге.

1. Проведите базовый EDA. Например, можно посмотреть сколько всего у вас диалогов, как распределены длины промптов и ответов, доступные инстурменты и так далее.
2. Подготовьте запросы, описания инструментов и эталоны, необходимые вашей функции награды. В отчете важно описать то, как вы подготавливаете данные для RL (это может быть раскручивание, как я описал выше, а может быть что угодно)
3. Разделите данные на трейн и валидацию. Тут важно правильно разделить – части одного диалога не должны попасть в разные сплиты. Можете также здесь оформить базовую фильтрацию, или что нибудь еще

4.2 Функция награды

Здесь вы можете реализовать что угодно. Как бейзлайн можно просто рассматривать полное соответствие того вызова, что сгенерировала LLM с тем вызовом, который был в датасете. Главное в отчете описать, что попробовали, какие были результаты и так далее. Также в отчете нужно показать примеры, как работает ваша функция награды.

4.3 Обучение модели

Напишите скрипт RL-обучения с использованием TRL.

Проведите не менее трёх различных конфигураций RL-обучения, не считая исходную модель. Конфигурации могут отличаться алгоритмом, наградой или хотя бы одним значимым гиперпараметром. Три разных алгоритма не обязательны.

Возможные направления экспериментов:

- форма loss, масштабирование advantage, token/sequence ratio;
- функция награды, веса её компонент, баланс примеров с вызовом и без вызова;
- learning rate, scheduler, warmup, число шагов и gradient clipping;
- batch size, gradient accumulation и число генераций на запрос (`num_generations`);
- число обновлений на собранных генерациях (`num_iterations`) и границы clipping;
- KL-коэффициент `beta`, температура sampling, максимальная длина ответа и обработка обрезанных генераций;
- rank LoRA, набор обучаемых модулей, состав и объём данных.

Сформулируйте гипотезу для каждого запуска, сохраните его конфигурацию, логи, результаты и так далее.

4.4 Бенчмаркинг

BFCL проверяет выбор функций и корректность аргументов. В домашке используется семпл примеров.

Категория	Что требуется	Примеров
Simple	Один вызов одной доступной функции	50
Multiple	Выбор одной функции из нескольких	150
Parallel	Несколько вызовов одной функции	50
Parallel multiple	Несколько вызовов при выборе из нескольких функций	50
Irrelevance	Ответ без вызова инструмента	50

Каждый пример получает 0 или 1 по правилам официального Python AST-checker. Вызовы разбираются и проверяются без исполнения функций. Для нескольких вызовов должен быть правильным весь набор; порядок самих вызовов не важен. В Irrelevance правильный ответ не содержит вызова. Пустые, обрезанные и некорректно размеченные ответы считаются ошибкой.

Оцените исходную модель и выбранный чекпоинт каждой из трёх (или более) конфигураций одним и тем же `evaluate.py`.

Не меняйте `evaluate.py`, `bfcl.jsonl`, токенизатор и параметры финальной генерации. В evaluator я отключаю ризонинг, ставлю температуру 0 и максимум 1024 новых токена. Для анализа используйте сохранённые ответы из `predictions.jsonl`.

4.5 Финальная таблица и отчёт

Подготовьте короткий отчёт с описанием данных (EDA, как вы с ними работали, раскидывали), формулой награды (опишите также и другие эксперименты), гипотезами экспериментов и выводами. Для каждого запуска укажите графика лосса, графики наград, используемую награду (формулку) изменённые гиперпараметры, выбранный шаг и результаты бенчмарка.

В итоговой таблице должны быть бейзлайн и минимум три RL-конфигурации. Пример структуры таблицы ниже

Запуск	Overall	Simple	Multiple	Parallel	Parallel + Multiple	Irrel.
Baseline	—	—	—	—	—	—
RL-1	—	—	—	—	—	—
RL-2	—	—	—	—	—	—
RL-3	—	—	—	—	—	—

Кратко ответьте на вопросы:

1. Почему выбрали такую награду?
2. Какое изменение сильнее всего повлияло на качество? Что позволяет отделить его эффект от остальных изменений?
3. Совпадает ли рост награды с ростом BFCL ? Если нет, какие примеры или кривые объясняют расхождение?
4. Что происходит с Advantage, если все ответы на запрос получают одинаковую награду?
5. Какие ошибки остались у лучшей модели? Разберите несколько примеров из разных категорий, включая лишний вызов и пропущенный нужный вызов.

5 Что сдавать

На GitHub закиньет следующее:

1. **Код и конфигурации:** подготовка данных, вычисление награды, обучение и экспорт модели. В README укажите команды воспроизведения, сид, версии данных и расположение результатов.
2. **Отчёт в PDF** с таблицами, графиками и ответами на вопросы.
3. **Результаты оценки:** `metrics.json` и `predictions.jsonl` для бейзлайна и всех конфигураций из итоговой таблицы.